

ЛАБОРАТОРНАЯ РАБОТА № 4

«Наследование. Обработка исключительных ситуаций»

Группа: РИМ-150950

Студент: Ляхова Елизавета Олеговна

Ссылка на github: https://github.com/Lizaswer/java_course_master.git

Цель: знакомство с иерархией классов исключений и получение навыков обработки ошибок.

Ход работы:

example1.java

```
package lrl4.examples;
```

```
public class example1 {  
    public static void main(String[] args) {  
        try {  
            System.out.println("0");  
            throw new RuntimeException("Непроверяемая  
ошибка");  
        } catch (RuntimeException e) {  
            System.out.println("1 " + e);  
        }  
        System.out.println("2");  
    }  
}
```

example2.java

```
package lrl4.examples;
```

```
public class example2 {  
    public static void main(String[] args) {  
        try {  
            System.out.println("0");  
            throw new RuntimeException("Непроверяемая  
ошибка");  
        } catch (Exception e) {  
            System.out.println("2 " + e);  
        }  
    }  
}
```

```
}
```

example3.java

```
package lrl4.examples;

public class example3 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new RuntimeException("ошибка");
        } catch (NullPointerException e) {
            System.out.println("1");
        } catch (RuntimeException e) {
            System.out.println("2");
        } catch (Exception e) {
            System.out.println("3");
        }
        System.out.println("4");
    }
}
```

example4.java

```
package lrl4.examples;

public class example4 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new RuntimeException("ошибка");
        } catch (NullPointerException e) {
            System.out.println("1");
        } catch (Exception e) {
            System.out.println("2");
        } catch (Error e) {
            System.out.println("3");
        }
    }
}
```

```

    }
    System.out.println("4");
}
}

```

example5.java

```

package lrl4.examples;

public class example5 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new RuntimeException("ошибка");
        } catch (NullPointerException e) {
            System.out.println("1");
        } catch (RuntimeException e) {
            System.out.println("Перехвачено: " +
e.getMessage());
        }
        System.out.println("2");
    }
}

```

Комментарий: исключение RuntimeException не перехватывается, программа аварийно завершается, добавлен перехват RuntimeException.

example6.java

```

package lrl4.examples;

public class example6 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new NullPointerException("ошибка");
        } catch (NullPointerException e) { //
ПОТОМОК

```

```

        System.out.println("1");
    } catch (RuntimeException e) { //
ПОТОМОК
        System.out.println("2");
    } catch (Exception e) { //
предок — в конце
        System.out.println("3");
    }
    System.out.println("4");
}
}

```

Комментарий: неправильный порядок catch-блоков — Exception (предок) раньше RuntimeException (потомка). Компилятор выдаёт ошибку, поменяли порядок — сначала потомки, потом предки.

example7.java

```

package lrl4.examples;

public class example7 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new NullPointerException("ошибка");
        } catch (NullPointerException e) {
            System.out.println("1");
            try {
                throw new ArithmeticException();
            } catch (ArithmeticException ex) { //
<- ДОБАВЛЕНО
                System.out.println("Перехвачено
второе исключение");
            }
        }
        System.out.println("3");
    }
}

```

Комментарий: В блоке catch выбрасывается новое исключение ArithmeticException, которое никто не перехватывает → программа падает, добавили обработку второго исключения.

example8.java

```
package lrl4.examples;

public class example8 {
    public static int m() {
        try {
            System.out.println("0");
            throw new RuntimeException();
        } finally {
            System.out.println("1");
        }
    }

    public static void main(String[] args) {
        System.out.println(m());
    }
}
```

example9.java

```
package lrl4.examples;

public class example9 {
    public static int m() {
        try {
            System.out.println("0");
            return 55;
        } finally {
            System.out.println("1");
        }
    }

    public static void main(String[] args) {
        System.out.println(m());
    }
}
```

```
}
```

example10.java

```
package lrl4.examples;

public class example10 {
    public static int m() {
        try {
            System.out.println("0");
            return 15;
        } finally {
            System.out.println("1");
            return 20;
        }
    }

    public static void main(String[] args) {
        System.out.println(m());
    }
}
```

example11.java

```
package lrl4.examples;

public class example11 {
    public static void main(String[] args) {
        try {
            System.out.println("0");
            throw new NullPointerException("ошибка");
        } catch (NullPointerException e) {
            System.out.println("1");
        } finally {
            System.out.println("2");
        }
        System.out.println("3");
    }
}
```

```
}  
}
```

example12.java

```
package lrl4.examples;  
  
public class example12 {  
    public static void m(String str, double chislo) {  
        if (str == null) {  
            throw new  
IllegalArgumentException("Строка введена неверно");  
        }  
        if (chislo > 0.001) {  
            throw new  
IllegalArgumentException("Неверное число");  
        }  
    }  
    public static void main(String[] args) {  
        try {  
            m(null, 0.000001);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Ошибка: " +  
e.getMessage());  
        }  
    }  
}
```

Комментарий: исключение выбрасывается, но не перехватывается → программа падает, добавлен try-catch в main.

example13.java

```
package lrl4.examples;  
  
public class example13 {  
    public static void main(String[] args) {  
        try {
```



```

        int l = args.length;
        System.out.println("размер массива= " +
l);
        int h = 10 / l;
        args[l + 1] = "10";
    } catch (ArithmeticException e) {
        System.out.println("Деление на ноль");
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("Индекс не
существует");
    }
}
}
}

```

example14.java

```

package lrl4.examples;

public class example14 {
    public static void m(int x) throws
ArithmeticException {
        int h = 10 / x;
    }

    public static void main(String[] args) {
        try {
            int l = args.length;
            System.out.println("размер массива= " +
l);
            m(l);
        } catch (ArithmeticException e) {
            System.out.println("Ошибка: Деление на
ноль");
        }
    }
}
}

```

Задание 2.1 Среднее значение среди положительных элементов массива

```
package lrl4.tasks;

import java.util.InputMismatchException;
import java.util.Scanner;

public class task1 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[] array = null;
        int size = 0;

        while (true) {
            try {
                System.out.print("Введите размер массива: ");
                size = scanner.nextInt();
                if (size <= 0) {
                    System.out.println("Размер должен быть положительным числом. Повторите ввод.");
                    continue;
                }
                array = new int[size];
                break;
            } catch (InputMismatchException e) {
                System.out.println("Ошибка: введена строка вместо числа. Повторите ввод.");
                scanner.next(); // очистка буфера
            }
        }

        for (int i = 0; i < size; i++) {
            while (true) {
                try {
                    System.out.print("array[" + i + "]" + " = ");
                    array[i] = scanner.nextInt();
                } catch (InputMismatchException e) {
                    System.out.println("Ошибка: введена строка вместо числа. Повторите ввод.");
                    scanner.next();
                }
            }
        }
    }
}
```

```
                break;
            } catch (InputMismatchException e) {
                System.out.println("Ошибка:
введена строка вместо числа. Введите целое число.");
                scanner.next(); // очистка буфера
            }
        }
    }

    int sum = 0;
    int positiveCount = 0;

    for (int value : array) {
        if (value > 0) {
            sum += value;
            positiveCount++;
        }
    }

    try {
        if (positiveCount == 0) {
            throw new Exception("Положительные
элементы отсутствуют.");
        }

        double average = (double) sum /
positiveCount;
        System.out.println("Среднее значение
среди положительных элементов: " + average);
    } catch (Exception e) {
        System.out.println("Ошибка: " +
e.getMessage());
    }

    scanner.close();
}
}
```

Задание 2.2 Вывод столбца матрицы по номеру

```
package lrl4.tasks;

import java.util.InputMismatchException;
import java.util.Scanner;

public class task2 {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        Scanner scanner = new Scanner(System.in);
        int columnNumber = -1;

        while (true) {
            try {
                System.out.print("Введите номер столбца (от 0 до " + (matrix[0].length - 1) + "): ");
                columnNumber = scanner.nextInt();

                if (columnNumber < 0 || columnNumber >= matrix[0].length) {
                    throw new
ArrayIndexOutOfBoundsException("Столбца с номером "
+ columnNumber + " не существует.");
                }
                break;
            } catch (InputMismatchException e) {
                System.out.println("Ошибка: введена строка вместо числа. Повторите ввод.");
                scanner.next(); // очистка буфера
            }
        }
    }
}
```

```

        } catch (ArrayIndexOutOfBoundsException
e) {
            System.out.println("Ошибка: " +
e.getMessage());
        }
    }

    System.out.println("Столбец " + columnNumber
+ ":");
    for (int i = 0; i < matrix.length; i++) {
        System.out.println("matrix[" + i + "][" +
columnNumber + "] = " + matrix[i][columnNumber]);
    }

    scanner.close();
}
}

```

Задание 2.3 Сумма элементов типа byte с контролем диапазона

```

package lrl4.tasks;

import java.util.InputMismatchException;
import java.util.Scanner;

public class task3 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        byte[] array = null;
        int size = 0;

        while (true) {
            try {
                System.out.print("Введите размер
массива: ");
                size = scanner.nextInt();
            }

```

```
        if (size <= 0) {
            System.out.println("Размер должен
быть положительным числом.");
            continue;
        }
        array = new byte[size];
        break;
    } catch (InputMismatchException e) {
        System.out.println("Ошибка: введена
строка вместо числа.");
        scanner.next();
    }
}

for (int i = 0; i < size; i++) {
    while (true) {
        try {
            System.out.print("array[" + i +
"] (от -128 до 127): ");
            int inputValue =
scanner.nextInt();

            if (inputValue < Byte.MIN_VALUE ||
inputValue > Byte.MAX_VALUE) {
                throw new Exception("Значение
" + inputValue + " выходит за границы типа byte
(диапазон от -128 до 127).");
            }

            array[i] = (byte) inputValue;
            break;
        } catch (InputMismatchException e) {
            System.out.println("Ошибка:
введена строка вместо числа.");
            scanner.next();
        } catch (Exception e) {
            System.out.println("Ошибка: " +
e.getMessage());
        }
    }
}
```

```

    }
}

int sum = 0;
boolean overflow = false;

for (byte value : array) {
    if (sum > 0 && value > Byte.MAX_VALUE -
sum) {
        overflow = true;
    }
    if (sum < 0 && value < Byte.MIN_VALUE -
sum) {
        overflow = true;
    }
    sum += value;
}

System.out.println("Массив: ");
for (byte value : array) {
    System.out.print(value + " ");
}
System.out.println();

if (overflow) {
    System.out.println("Внимание: при
сложении произошло переполнение. Сумма может быть
некорректной.");
}

try {
    if (sum < Byte.MIN_VALUE || sum >
Byte.MAX_VALUE) {
        throw new Exception("Сумма " + sum +
" выходит за границы типа byte.");
    }
}

```

```

        System.out.println("Сумма элементов
(int): " + sum);
        System.out.println("Сумма элементов
(byte): " + (byte) sum);
    } catch (Exception e) {
        System.out.println("Ошибка: " +
e.getMessage());
        System.out.println("Сумма в виде int: " +
sum);
    }

    scanner.close();
}
}

```

Задачи с timus:

Задача 1000

```

package lrl4.timus;

import java.io.*;
import java.util.*;

public class task1000 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int a = scanner.nextInt();
        int b = scanner.nextInt();
        System.out.println(a + b);
        scanner.close();
    }
}

```

Задача 1068


```

package lrl4.timus;

import java.util.Scanner;

public class task1068 {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int sum;

        if (n > 0) {
            sum = n * (n + 1) / 2;
        } else {
            sum = (Math.abs(n) + 2) * (Math.abs(n) +
1) / 2;
            sum = -sum + 1;
        }

        System.out.println(sum);
        scanner.close();
    }
}

```

Вывод: В ходе выполнения лабораторной работы были изучены принципы наследования и иерархия исключений в Java. Воспроизведены и исправлены 14 примеров, демонстрирующих работу конструкций try-catch-finally, throw и throws. Разработаны три программы с полной обработкой исключительных ситуаций (некорректный ввод, выход за границы диапазона, отсутствие данных, переполнение). Решены две задачи с сайта Timus. В результате получены практические навыки создания отказоустойчивых программ на Java.